PersonalFinanceManager

PHASE 2 — DATABASE DESIGN + DATA DICTIONARY + ER DIAGRAM + SQL SERVER PLAN

Purpose: A personal finance application for managing monthly salary, expenses, savings, investments, bills and financial goals.

Item	Design
Database	PersonalFinanceManager_DB
Platform	SQL Server
Application	ASP.NET Core MVC
API	ASP.NET Core Web API
Data Access	ADO.NET
Language	C#
Primary Use	Personal monthly money management

This document follows the organization and level of detail of the supplied BusinessManagementERP Phase 2 reference, but its entities and rules are redesigned specifically for a personal finance manager.

1. Database Objectives

The application should answer: “I received my monthly income; where should my money go, where did it actually go, and how much is left?”

The project is intentionally personal. It does not need ERP entities such as customers, suppliers, warehouses, branches, sales orders or inventory.

The database supports income, bank/cash accounts, categories, daily transactions, monthly plans, savings goals, investments and bills. Authentication and audit history are included as learning-oriented supporting features.

Example monthly situation:

Monthly Item	Planned	Actual / Current
Income	₹50,000	₹50,000
Expenses	₹20,000	₹17,500
Savings	₹10,000	₹10,000
Investments	₹5,000	₹5,000
Remaining	₹15,000	₹17,500

The exact allocation is your choice. The application tracks your own plan instead of forcing a fixed financial rule.

2. Core Business Rules

Rule	Decision
Personal ownership	Each financial record belongs to the user account.
Money precision	Use decimal(18,2) for currency values.
Positive amounts	Amounts are positive; TransactionType determines Income or Expense.
Financial history	Do not casually hard-delete financial transactions.
Monthly planning	MonthlyPlans stores the amount you intend to allocate to a category for a month.
Actual spending	Transactions represent what actually happened.
Plan vs actual	Dashboard compares MonthlyPlans with Transactions.
Accounts	Accounts represent where money is held: bank, cash, wallet, card, etc.
Savings goals	SavingsGoals represent a target and current progress.
Bills	Bills represent recurring/expected payments and payment status.
Investments	Investments track invested amount and current tracked value.

These principles retain the reference document's useful database ideas—PK/FK integrity, decimal money precision, audit timestamps and protecting financial history—while removing business-ERP complexity.

3. Main Entity Groups

Group	Tables	Purpose
Identity	Users	Login and application owner
Income	Income	Salary and other earnings
Account Management	Accounts	Bank, cash, wallet, card balances
Transaction Management	Categories, Transactions, TransactionAttachments	Classify and record daily money movement
Planning	MonthlyPlans	Personal monthly allocation
Savings	SavingsGoals	Targets such as laptop/emergency fund
Investment	Investments	SIP, mutual funds, stocks, FD tracking
Bills	Bills	Recurring obligations and due dates
Platform	Notifications, AuditLogs	Alerts and change history

Core Version 1: Users → Income → Accounts → Categories → Transactions → MonthlyPlans → SavingsGoals → Investments → Bills

4. Data Dictionary — Users

Field	SQL Type	Key / Constraint	Null?	Purpose
UserId	int	PK Identity	N	Application user
FullName	nvarchar(150)		N	User name
Email	nvarchar(150)	Unique	N	Login email
PasswordHash	nvarchar(500)		N	Hashed password only
Phone	nvarchar(30)		Y	Phone number
IsActive	bit	Default 1	N	Account status
CreatedAtUtc	datetime2	Default UTC	N	Created time

4. Data Dictionary — Income

Field	SQL Type	Key / Constraint	Null?	Purpose
IncomeId	bigint	PK Identity	N	Income record
UserId	int	FK Users	N	Owner
IncomeSource	nvarchar(100)		N	Salary/Freelance/etc.
Amount	decimal(18,2)		N	Income amount
IncomeType	nvarchar(30)		N	Salary/Bonus/Freelance/Other
IncomeDate	date		N	Date received
Description	nvarchar(300)		Y	Optional note
CreatedAtUtc	datetime2	Default UTC	N	Created time

4. Data Dictionary — Accounts

Field	SQL Type	Key / Constraint	Null?	Purpose
AccountId	int	PK Identity	N	Money account
UserId	int	FK Users	N	Owner
AccountName	nvarchar(100)		N	HDFC/Cash/Wallet/etc.
AccountType	nvarchar(40)		N	Bank/Cash/Wallet/Card
OpeningBalance	decimal(18,2)		N	Initial balance
CurrentBalance	decimal(18,2)		N	Current displayed balance
IsActive	bit	Default 1	N	Status
CreatedAtUtc	datetime2	Default UTC	N	Created time

4. Data Dictionary — Categories

Field	SQL Type	Key / Constraint	Null?	Purpose
CategoryId	int	PK Identity	N	Category
UserId	int	FK Users	N	Owner
CategoryName	nvarchar(100)		N	Food/Rent/Salary/etc.
CategoryType	nvarchar(20)	Income/Expense/Saving/Investment	N	Category purpose
ParentCategoryId	int	Self FK	Y	Optional hierarchy
IsActive	bit	Default 1	N	Status

4. Data Dictionary — Transactions

Field	SQL Type	Key / Constraint	Null?	Purpose
TransactionId	bigint	PK Identity	N	Daily financial movement
UserId	int	FK Users	N	Owner
AccountId	int	FK Accounts	N	Account used
CategoryId	int	FK Categories	N	Classification
Amount	decimal(18,2)		N	Amount
TransactionType	nvarchar(20)	Income/Expense	N	Direction
TransactionDate	datetime2		N	When it happened
Description	nvarchar(500)		Y	Description
CreatedAtUtc	datetime2	Default UTC	N	Created time

4. Data Dictionary — MonthlyPlans

Field	SQL Type	Key / Constraint	Null?	Purpose
MonthlyPlanId	int	PK Identity	N	Monthly plan row
UserId	int	FK Users	N	Owner
PlanYear	int		N	Plan year
PlanMonth	int	1–12	N	Plan month
CategoryId	int	FK Categories	N	Planned category
PlannedAmount	decimal(18,2)		N	Your planned amount
CreatedAtUtc	datetime2	Default UTC	N	Created time

4. Data Dictionary — SavingsGoals

Field	SQL Type	Key / Constraint	Null?	Purpose
GoalId	int	PK Identity	N	Savings goal
UserId	int	FK Users	N	Owner
GoalName	nvarchar(150)		N	Laptop/Emergency Fund/etc.
TargetAmount	decimal(18,2)		N	Goal target
CurrentAmount	decimal(18,2)		N	Amount saved
TargetDate	date		Y	Target date
Status	nvarchar(30)		N	Active/Completed/Cancelled
CreatedAtUtc	datetime2	Default UTC	N	Created time

4. Data Dictionary — Investments

Field	SQL Type	Key / Constraint	Null?	Purpose
InvestmentId	int	PK Identity	N	Investment record
UserId	int	FK Users	N	Owner
InvestmentType	nvarchar(50)		N	SIP/Stock/MF/FD/etc.
InvestmentName	nvarchar(150)		N	Investment name
InvestedAmount	decimal(18,2)		N	Total invested
CurrentValue	decimal(18,2)		N	Current tracked value
InvestmentDate	date		N	Investment date
Description	nvarchar(300)		Y	Optional note

4. Data Dictionary — Bills

Field	SQL Type	Key / Constraint	Null?	Purpose
BillId	int	PK Identity	N	Bill
UserId	int	FK Users	N	Owner
BillName	nvarchar(150)		N	Rent/Internet/etc.
CategoryId	int	FK Categories	N	Expense category
Amount	decimal(18,2)		N	Expected amount
DueDate	date		N	Due date
Frequency	nvarchar(30)		N	Monthly/Yearly/etc.
IsPaid	bit	Default 0	N	Payment status
IsActive	bit	Default 1	N	Status

4. Data Dictionary — Supporting Tables

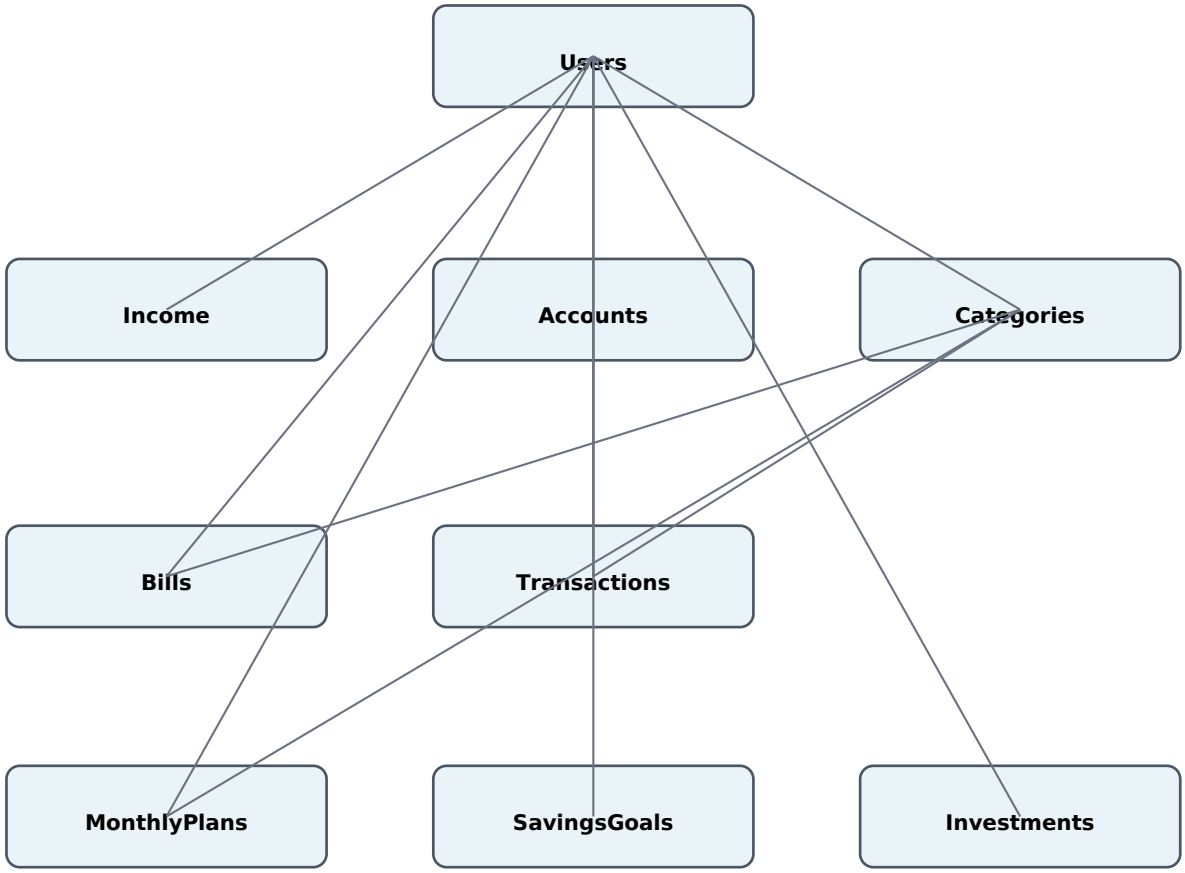
Field	SQL Type	Key / Constraint	Null?	Purpose
AttachmentId	bigint	PK Identity	N	Receipt attachment
TransactionId	bigint	FK Transactions	N	Transaction owner
FileName	nvarchar(255)		N	Uploaded file name
FilePath	nvarchar(500)		N	Stored path
UploadedAtUtc	datetime2		N	Upload time
NotificationId	bigint	PK Identity	N	Notification
UserId	int	FK Users	N	Recipient
NotificationType	nvarchar(40)		N	Budget/Bill/etc.
Title	nvarchar(200)		N	Notification title
Message	nvarchar(1000)		N	Notification message
IsRead	bit	Default 0	N	Read status

4. Data Dictionary — AuditLogs

Field	SQL Type	Key / Constraint	Null?	Purpose
AuditLogId	bigint	PK Identity	N	Audit event
UserId	int	FK Users	N	Actor
EntityName	nvarchar(100)		N	Affected entity/table
EntityId	nvarchar(50)		N	Affected record
Action	nvarchar(30)		N	Create/Update/etc.
OldValues	nvarchar(max)		Y	Before snapshot
NewValues	nvarchar(max)		Y	After snapshot
CreatedAtUtc	datetime2		N	Event time

5. ER Diagram — Logical Relationship Map

Main relationship: one user can have many financial records. Transactions connect the user to an account and category. Monthly plans and bills also use categories.



Readable ERD: Users → Income / Accounts / Categories / Transactions / MonthlyPlans / SavingsGoals / Investments / Bills

Accounts → Transactions; Categories → Transactions / MonthlyPlans / Bills

6. SQL Server Starter Script

```
CREATE DATABASE PersonalFinanceManager_DB; GO USE PersonalFinanceManager_DB; GO

CREATE TABLE Users ( UserId INT IDENTITY(1,1) NOT NULL PRIMARY KEY, FullName NVARCHAR(150) NOT NULL, Email NVARCHAR(150) NOT NULL UNIQUE, PasswordHash NVARCHAR(500) NOT NULL, Phone NVARCHAR(30) NULL, IsActive BIT NOT NULL DEFAULT 1, CreatedAtUtc DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME() ); GO

CREATE TABLE Accounts ( AccountId INT IDENTITY(1,1) NOT NULL PRIMARY KEY, UserId INT NOT NULL, AccountName NVARCHAR(100) NOT NULL, AccountType NVARCHAR(40) NOT NULL, OpeningBalance DECIMAL(18,2) NOT NULL DEFAULT 0, CurrentBalance DECIMAL(18,2) NOT NULL DEFAULT 0, IsActive BIT NOT NULL DEFAULT 1, CreatedAtUtc DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(), CONSTRAINT FK_Accounts_Users FOREIGN KEY (UserId) REFERENCES Users(UserId) ); GO

CREATE TABLE Categories ( CategoryId INT IDENTITY(1,1) NOT NULL PRIMARY KEY, UserId INT NOT NULL, CategoryName NVARCHAR(100) NOT NULL, CategoryType NVARCHAR(20) NOT NULL, ParentCategoryId INT NULL, IsActive BIT NOT NULL DEFAULT 1, CONSTRAINT FK_Categories_Users FOREIGN KEY (UserId) REFERENCES Users(UserId), CONSTRAINT FK_Categories_Parent FOREIGN KEY (ParentCategoryId) REFERENCES Categories(CategoryId), CONSTRAINT CK_Categories_Type CHECK (CategoryType IN ('Income','Expense','Saving','Investment')) ); GO
```

6. SQL Server Starter Script — Financial Core

```
CREATE TABLE Income ( IncomeId BIGINT IDENTITY(1,1) NOT NULL PRIMARY KEY, UserId INT NOT NULL, IncomeSource NVARCHAR(100) NOT NULL, Amount DECIMAL(18,2) NOT NULL, IncomeType NVARCHAR(30) NOT NULL, IncomeDate DATE NOT NULL, Description NVARCHAR(300) NULL, CreatedAtUtc DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(), CONSTRAINT FK_Income_Users FOREIGN KEY (UserId) REFERENCES Users(UserId), CONSTRAINT CK_Income_Amount CHECK (Amount > 0) ); GO

CREATE TABLE Transactions ( TransactionId BIGINT IDENTITY(1,1) NOT NULL PRIMARY KEY, UserId INT NOT NULL, AccountId INT NOT NULL, CategoryId INT NOT NULL, Amount DECIMAL(18,2) NOT NULL, TransactionType NVARCHAR(20) NOT NULL, TransactionDate DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(), Description NVARCHAR(500) NULL, CreatedAtUtc DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(), CONSTRAINT FK_Transactions_Users FOREIGN KEY (UserId) REFERENCES Users(UserId), CONSTRAINT FK_Transactions_Accounts FOREIGN KEY (AccountId) REFERENCES Accounts(AccountId), CONSTRAINT FK_Transactions_Categories FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId), CONSTRAINT CK_Transactions_Amount CHECK (Amount > 0), CONSTRAINT CK_Transactions_Type CHECK (TransactionType IN ('Income','Expense')) ); GO

CREATE TABLE MonthlyPlans ( MonthlyPlanId INT IDENTITY(1,1) NOT NULL PRIMARY KEY, UserId INT NOT NULL, PlanYear INT NOT NULL, PlanMonth INT NOT NULL, CategoryId INT NOT NULL, PlannedAmount DECIMAL(18,2) NOT NULL, CreatedAtUtc DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(), CONSTRAINT FK_MonthlyPlans_Users FOREIGN KEY (UserId) REFERENCES Users(UserId), CONSTRAINT FK_MonthlyPlans_Categories FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId), CONSTRAINT CK_MonthlyPlans_Month CHECK (PlanMonth BETWEEN 1 AND 12), CONSTRAINT CK_MonthlyPlans_Amount CHECK (PlannedAmount >= 0), CONSTRAINT UQ_MonthlyPlans UNIQUE (UserId, PlanYear, PlanMonth, CategoryId) ); GO

CREATE TABLE SavingsGoals ( GoalId INT IDENTITY(1,1) NOT NULL PRIMARY KEY, UserId INT NOT NULL, GoalName NVARCHAR(150) NOT NULL, TargetAmount DECIMAL(18,2) NOT NULL, CurrentAmount DECIMAL(18,2) NOT NULL DEFAULT 0, TargetDate DATE NULL, Status NVARCHAR(30) NOT NULL DEFAULT 'Active', CreatedAtUtc DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(), CONSTRAINT FK_SavingsGoals_Users FOREIGN KEY (UserId) REFERENCES Users(UserId), CONSTRAINT CK_SavingsGoals_Target CHECK (TargetAmount > 0) ); GO
```

6. SQL Server Starter Script — Investments and Bills

```
CREATE TABLE Investments ( InvestmentId INT IDENTITY(1,1) NOT NULL PRIMARY KEY, UserId INT NOT NULL, InvestmentType NVARCHAR(50) NOT NULL, InvestmentName NVARCHAR(150) NOT NULL, InvestedAmount DECIMAL(18,2) NOT NULL, CurrentValue DECIMAL(18,2) NOT NULL, InvestmentDate DATE NOT NULL, Description NVARCHAR(300) NULL, CONSTRAINT FK_Investments_Users FOREIGN KEY (UserId) REFERENCES Users(UserId) ); GO

CREATE TABLE Bills ( BillId INT IDENTITY(1,1) NOT NULL PRIMARY KEY, UserId INT NOT NULL, BillName NVARCHAR(150) NOT NULL, CategoryId INT NOT NULL, Amount DECIMAL(18,2) NOT NULL, DueDate DATE NOT NULL, Frequency NVARCHAR(30) NOT NULL, IsPaid BIT NOT NULL DEFAULT 0, IsActive BIT NOT NULL DEFAULT 1, CONSTRAINT FK_Bills_Users FOREIGN KEY (UserId) REFERENCES Users(UserId), CONSTRAINT FK_Bills_Categories FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId) ); GO

CREATE INDEX IX_Transactions_User_Date ON Transactions(UserId, TransactionDate); GO CREATE INDEX IX_Transactions_Category_Date ON Transactions(CategoryId, TransactionDate); GO CREATE INDEX IX_Income_User_Date ON Income(UserId, IncomeDate); GO CREATE INDEX IX_Bills_User_DueDate ON Bills(UserId, DueDate); GO
```

7. Important Reporting Queries

```
-- Total income for a month SELECT ISNULL(SUM(Amount),0) AS TotalIncome FROM Income WHERE UserId = @UserId AND IncomeDate >=
@MonthStart AND IncomeDate < @NextMonth;

-- Total expenses for a month SELECT ISNULL(SUM(Amount),0) AS TotalExpense FROM Transactions WHERE UserId = @UserId AND
TransactionType = 'Expense' AND TransactionDate >= @MonthStart AND TransactionDate < @NextMonth;

-- Category-wise expense SELECT C.CategoryName, SUM(T.Amount) AS TotalExpense FROM Transactions T INNER JOIN Categories C ON
T.CategoryId = C.CategoryId WHERE T.UserId = @UserId AND T.TransactionType = 'Expense' AND T.TransactionDate >= @MonthStart AND
T.TransactionDate < @NextMonth GROUP BY C.CategoryName ORDER BY TotalExpense DESC;

-- Plan vs actual SELECT C.CategoryName, MP.PlannedAmount, ISNULL(SUM(T.Amount),0) AS ActualAmount, MP.PlannedAmount -
ISNULL(SUM(T.Amount),0) AS Difference FROM MonthlyPlans MP INNER JOIN Categories C ON MP.CategoryId = C.CategoryId LEFT JOIN
Transactions T ON T.CategoryId = MP.CategoryId AND T.UserId = MP.UserId AND T.TransactionType = 'Expense' AND T.TransactionDate
>= DATEFROMPARTS(MP.PlanYear, MP.PlanMonth, 1) AND T.TransactionDate < DATEADD(MONTH, 1, DATEFROMPARTS(MP.PlanYear,
MP.PlanMonth, 1)) WHERE MP.UserId = @UserId AND MP.PlanYear = @Year AND MP.PlanMonth = @Month GROUP BY C.CategoryName,
MP.PlannedAmount;
```

8. Personal Finance Dashboard

The dashboard is the main reason for building this application: it converts your raw transactions into a clear picture of your month.

Metric	Example
Monthly Income	₹50,000
Planned Expenses	₹20,000
Actual Expenses	₹17,500
Savings	₹10,000
Investments	₹5,000
Remaining	₹17,500

Plan vs Actual

Category	Planned	Actual	Difference
Food	₹5,000	₹4,200	₹800 left
Travel	₹3,000	₹2,700	₹300 left
Shopping	₹3,000	₹4,200	₹1,200 over
Savings	₹10,000	₹10,000	On plan

The system should not decide your percentages for you. You enter your own MonthlyPlan and the application tells you whether your actual spending is following that plan.

9. MVC + ADO.NET + Web API Architecture

Layer	Responsibility
SQL Server	Tables, relationships, constraints, indexes, stored procedures
ADO.NET Repository	SqlConnection, SqlCommand, SqlDataReader, parameters
Service Layer	Business rules and calculations
Web API	REST endpoints and DTOs
ASP.NET Core MVC	Controllers, Views, forms and dashboard
C#	Models, DTOs, services, validation

Example: Add Expense

View → MVC Controller → Service → ADO.NET Repository → SQL → SQL Server → Response → View

The Web API can expose the same business capabilities for a mobile app, JavaScript client or future external client.

10. Development Phases

Phase	Build
1	Create database and tables
2	Insert practice data; learn SELECT/JOIN/GROUP BY
3	Create C# models
4	Build ADO.NET connection and repository
5	Create Web API CRUD endpoints
6	Create ASP.NET Core MVC project
7	Income, Accounts, Categories and Transactions CRUD
8	Monthly Plan and Plan vs Actual
9	Savings Goals, Investments and Bills
10	Dashboard and reports
11	Authentication/authorization
12	Validation, logging, error handling and testing

Start small: the first usable version should focus on Income → Accounts → Categories → Transactions → MonthlyPlans → Dashboard. Add Goals, Investments and Bills after that core workflow is stable.

11. Version 1 Sign-Off Checklist

☐ Database created

☐ Users table created

☐ Income table created

☐ Accounts table created

☐ Categories table created

☐ Transactions table created

☐ MonthlyPlans table created

☐ SavingsGoals table created

☐ Investments table created

☐ Bills table created

☐ PK/FK relationships verified

☐ Money columns use decimal(18,2)

☐ Financial history protected from casual deletion

☐ Dashboard queries tested

☐ Plan vs Actual report tested

Reference design basis

The structure of this document is intentionally modeled after the supplied BusinessManagementERP Phase 2 PDF: objectives, rules, entity groups, data dictionary, relationship map, SQL plan, indexes/delete strategy and implementation plan. The business entities themselves are intentionally changed to fit a personal finance use case.